Prototype bring-up

For: the person at the bench with the assembled prototype board. **You need:** - a bench supply with an adjustable current limit, 0–20 V - a multimeter - a 100 MHz oscilloscope with a probe rated **300 V CAT II or better** (the boost rail reaches 116 V) - an ST-LINK V3 or J-Link with the 10-pin 1.27 mm Cortex cable - dummy loads for steps 4 and 5 - a 12 V automotive relay (or a 160 Ω, 2 W resistor) for step 6

Power the board up **one supply at a time**, in the order below. The board follows Bosch's pinout (decisions log §18), which has **no separate logic feed**: - **Pin 04** feeds the electronics *and* the injector stage, so the 100 V boost rail comes up with the logic. - **Pin 06** feeds the EGR bridge.

On the engine both pins come from the main relay, which the board closes itself (step 6). On the bench, feed them straight from the supply. Don't skip a step. If a reading is outside its range, **stop** and find out why before going on.

From step 2 on, TP12 is at about 100 V, because the boost converter's enable is tied on in copper and it starts as soon as pin 04 is powered. **The boost capacitors stay charged after the supply is off.** TP12 can hold close to 100 V for minutes. Before you touch the board after any powered step, measure TP12 to GND and wait until it reads **under 5 V**.

Expected values come from the production simulation blocks (`sim/blocks/`) and the sheet notes; each is named.

Where things are

Terminal	Edge	Position: signal (production pin)
J1001	left, top	1, 2: V_BAT_1R (04: electronics and injectors; use both above 5 A) · 3: V_BAT_2R (06, EGR) · 4: GND
J1002	left	1, 2: GND, the power return (Bosch pins 01/02)
J1003	left	injectors: INJ_A_03, INJ_B_05, INJ_LS1_73, INJ_LS2_29, INJ_LS3_07
J1004	top	MU_RETURN (88), EGR_HIGH_59, EGR_LOW_81, RLY_MAIN_OUT (50), RLY_BUZZER_OUT (69)
J1005	right	sensors: 5 excitations, 6 sensor inputs, SGND 34/36, GND (rail sensor return, 08)
J1006	right	speed: CRANK_HI_52, CRANK_LO_74, CAM_FREQ_46, CAM_GND_44, SHIELD_30, GND
J1007	bottom	1–5: switches 43/23/20/24/71 (ignition) · 6: water in fuel (21) · 7: diagnosis request (14) · 8: stop switch (25) · 9: GCU sync, analog (09)
J1008	bottom	CAN1_H, CAN1_L, GND, CAN2_H, CAN2_L, GND
J1009	inside, by J1005	2 × 10 bridge header: crank pair, cam, 6 sensor inputs, ignition on the odd pins; GND on the even pins
J301	by the MCU	SWD (Cortex 10-pin). SWO is wired to pin 6

Terminal	Edge	Position: signal (production pin)
J302	by the MCU	USB-C, **data only**: it does not power the board

Test points (TP1–TP24):

TP	Net	TP	Net
TP1	VBAT_PROT	TP13, TP14	GND (injector)
TP2	VBAT_INJ	TP15, TP16	ISNS_INJ_A, ISNS_INJ_B
TP3	VBAT_EGR	TP17, TP18	ISNS_MU, ISNS_EGR
TP4	12V_GATE	TP19, TP20	CRANK_EDGE, CAM_EDGE
TP5	5V_MAIN	TP21, TP22	CAN1_H, CAN1_L
TP6, TP11	GND	TP23, TP24	CAN2_H, CAN2_L
TP7	3V3_MCU	**TP12**	**BOOST_100V: up to 116 V. HV.**
TP8–TP10	5V_SENSOR_A/B/C		

1. Before any power

1. **Look** at every IC for orientation and solder bridges, under a loupe. Check the LQFP-100 (U301) and the HVSSOP boost controller (U401) closely.
2. **Resistance to GND** with the meter. Nothing on this list should read below 100 Ω. A short here would destroy parts at power-up.

Probe	Expect
TP1, TP2, TP3 (the feeds)	> 1 kΩ
TP5 (5V_MAIN), TP7 (3V3_MCU)	> 100 Ω (charging capacitors: the reading climbs)
TP12 (BOOST_100V)	> 10 kΩ, climbing (two 82 µF capacitors charging)
TP21–TP22, TP23–TP24 (H to L of each bus)	**120 Ω**, the split termination 60R0 + 60R0 (`can` sheet)

2. Pin 04: the electronics and the boost rail, limited to 1 A

Supply **13.5 V on J1001-1**, with GND on J1002, at a **1 A** limit. Leave pin 06 (J1001-3) open. Put the HV probe on TP12 before you switch on.

Check	Expect	Source
Supply current	at the limit for a moment while the boost capacitors charge, then well under it. **If it stays at the limit, switch off**	—
TP1 VBAT_PROT	about 13.5 V, less one FET drop	`reverse_battery.cir`
TP2 VBAT_INJ	about 13.5 V	`power_path.cir`
TP5 5V_MAIN	5.0 V, ripple under 50 mV p-p	`buck_preregulator.cir`
TP7 3V3_MCU	3.30 V. **It must sit inside 3.143–3.459 V**, or the supervisor resets the board. The TLV767 holds 1 %	`rails` sheet; `supervisor.cir`
TP8–TP10 5V_SENSOR_A/B/C	about 5.0 V, following 5V_MAIN	`sensor_rail.cir`

Check	Expect	Source
D303 (3V3 LED)	on	—
TP12 BOOST_100V (HV probe)	**99.7 V nominal**, never above about 102 V	`boost_converter.cir` ; `injector` sheet
`BOOST_MON` (PB11)	**2.43 V at 100 V** (24.9 k / (2 × 499 k + 24.9 k) = 0.0243)	`prototype/additions.py`
TP4 12V_GATE	about 12.3 V (a 13 V zener-referenced follower from the boost rail). The drivers' UVLO is 9.8 V	`injector` sheet

Then connect the debugger to J301. **SWD must connect,** and the debugger should read the device ID (STM32G474, `0x469`). Flash a test build that toggles `LED_STATUS` (PB7): D304 should blink.

The supervisor will reset the MCU every few tens of milliseconds unless the firmware kicks `WDT_KICK` (PC11) at 5–20 ms intervals (decisions log 11). With no firmware, a reset pulse train on NRST is correct behaviour, not a fault.

3. CAN and the sensor inputs

1. **CAN:** connect a USB-CAN adapter to J1008: **250 kbit/s on J1008-1/2** (pins 55/77, J1939) and **1000 kbit/s on J1008-4/5** (pins 87/86, the service link; Bosch's rates). Each bus is split-terminated on the board; terminate the adapter end too. Send and receive a frame on each bus.
2. **Sensors:** put a 10 kΩ potentiometer between each excitation and its SGND on J1005, with the wiper on the matching input. Sweep each one and read the ADC. - **The divided channels read about 0–3.07 V for 0–5 V in.** `sensor_ratiometric.cir` checks 5.0 V at the pad giving 3.072 V.
3. **Battery monitor:** `VBAT_MON` (PB12) should read **1.04 V at 13.5 V** (120 k / 10 k).

4. Pin 06 (EGR), then firing the injectors

Pin 06: connect 13.5 V to J1001-3 at a 1 A limit. - TP3 VBAT_EGR should read about 13.5 V. - Load the EGR outputs (J1004-2/-3) with a 10 Ω, 25 W resistor. - The EGR bridge stays coasting until firmware raises `EGR_EN`.

Injectors: put a dummy load on each injector output in J1003: 0.2 Ω + 200 µH, or a real injector on a bench.

Before firing any injector, raise the pin 04 limit to 5 A. The boost recharge after each shot draws more than 1 A; at a 1 A limit the supply folds back and the readings below are wrong.

With firmware firing one injection at 1 ms: - **TP15/TP16** (ISNS_INJ) should show the current rising to the 18 A peak in about **38 µs**, from the boost rail (`injector_boost.cir`). - After turn-off, the current should fall in about **36 µs** (`injector_turnoff.cir`). - **BOOST_100V** should be back at its setpoint before the next cylinder, 26.67 ms later.

5. The watchdog-kill test

1. With pins 04 and 06 powered and the firmware driving the outputs, **stop kicking** `WDT_KICK`.
2. At the end of the watchdog window, the TPS3850 asserts `WDO`. `WDO` pulls `RESET` low through D302 (a BAT54Q Schottky, about 0.3 V), so the MCU resets and Q301 turns off. `GATE_KILL` rises to about **12 V** (zener-clamped, 12.7 V maximum). Probe it at R305/Q301's drain.
3. **Every gate must drop:** - injector high and low sides - metering FET - EGR bridge

The kill FETs that do this are Q402–Q405, Q413, Q415, Q417, Q502 and Q507. The relays are not on `GATE_KILL`. The buzzer relay drops because the firmware is in reset. **The main relay stays closed while the ignition is on**, because the wake FET holds it, so on the engine the board keeps its supply through a reset.

Check the current on each dummy load, and the gate of Q412 (low side 1) on the scope. 4. On the scope, `WDT_FAULT` (PC12) and NRST go low together and come back high together. `WDT_FAULT` is high again by the time firmware runs, so it does not record which trip it was. **History:** RESET and `WDT_FAULT` were shorted on the production schematic until 15 Oct 2026; that fix removed the only path from `WDO` to the reset, and D302 (19 Oct) puts it back.

6. The ignition wake and the main relay (Bosch wiring)

On the engine the board has no supply until **pin 71 (ignition)** closes the main relay through the wake FET on pin 50. The firmware then holds the relay through the after-run (decisions log §18).

1. Board powered as in step 2, firmware running. Connect the relay coil (or the 160 Ω resistor) from **+13.5 V to J1004-4** (`RLY_MAIN_OUT`, pin 50).
2. **Ignition off (J1007-5 open), firmware not yet holding the relay:** J1004-4 sits at **13.5 V**. No coil current flows, so the relay is open. This is the sneak path the TVS replaced the diode to remove: there must be no current here.
3. **Ignition on (13.5 V on J1007-5):** J1004-4 falls to **under 0.1 V** and the relay pulls in. **Repeat with the MCU held in reset** (NRST to GND through the debugger): it must still pull in, because the wake FET closes it in hardware.
4. **Ignition off with the firmware running:** the relay stays in. The firmware saves the fault memory, waits for the engine to be stopped (on the bench it already is), then releases it, within about 10 s. `afterrun` reads 1 until then.

5. **Optional: a real main relay.** Wire its contact from the bench supply to J1001-1, -2 and -3, the way the engine does, and repeat 3–4. The board should start on ignition, and switch itself off after ignition off.
6. **Inputs:** 13.5 V on J1007-6 (water in fuel) for over 2 s raises an amber DTC, SPN 97. Grounding J1007-8 (stop switch) shuts the firmware down. J1007-9 (GCU sync) reads 0–10 V as 0–3.125 V on PB2.

Pass: all six steps within their ranges, and TP12 under 5 V before you disconnect. Record the readings in `docs/status.md` with the date and board serial.